

Algorithmique et Programmation orientée-objet

Ch.4 – Héritage

Bruno Quoitin
(bruno.quoitin@umons.ac.be)

Agenda

- **Objectifs du Chapitre**

- Découvrir deux des principes les plus importants de la Programmation orientée-objet
 - Héritage et re-définition de méthodes
 - Polymorphisme
- Comprendre plusieurs aspects techniques
 - Liaison dynamique
 - Run-time Type Identification

Table des Matières

- **Introduction**
 - Exemple introductif
 - Relation entre classes
- **Héritage en Java**
 - Déclaration de classe
 - Redéfinition de méthode
- **Classe Object**
- **Polymorphisme**
 - Transtypage
 - Liaison dynamique
- **Encapsulation revisitée**

Introduction

- **Notion d'héritage**

- La notion d'**héritage** est une des notions les plus importantes de la programmation orientée-objet. Elle consiste à créer de nouvelles classes d'objets à partir d'autres classes, par un processus de spécialisation.
 - Les classes nouvellement créées **héritent** des données et méthodes de leurs classes « parents ».
 - Extension : De nouvelles données et de nouveaux comportements peuvent être ajoutés.
 - Modifications : Les comportements hérités peuvent être modifiés (**redéfinition de méthodes**).
- Pour le programmeur, l'héritage permet de réduire la duplication de code. La définition rapide de nouvelles classes est possible sans « *copier-coller* ».

Héritage

- Exemple introductif

```
public class Livre {  
    public final String titre;  
    public final String auteur;  
    private int pages;  
  
    public Livre(String titre, String auteur, int pages) {  
        this.titre = titre;  
        this.auteur = auteur;  
        this.pages = pages;  
    }  
  
    public int getNumPages() {  
        return pages;  
    }  
  
    public String info() {  
        return "\"" + titre + "\" de " + auteur);  
    }  
}
```

Héritage

- Exemple introductif

Héritage : la classe Dico est basée sur la classe Livre.

Extension : nouveaux attributs et méthodes

```
public class Dico extends Livre {  
    private int defs;  
  
    public Dico(String titre, String auteur, int pages,  
                int defs) {  
        super(titre, auteur, pages);  
        this.defs = defs;  
    }  
  
    public int getNumDefs() {  
        return defs;  
    }  
  
    public String info() {  
        return "\"" + titre + "\" de " + auteur +  
            " (" + defs + " définitions)";  
    }  
}
```

Re-définition : constructeur et méthodes

Héritage

- Exemple introductif

```
public class TestLivres {  
  
    public static void main(String[] args) {  
        Livre javaConcepts = new Livre("Java Concepts",  
                                         "Cay Horstmann", 500);  
        System.out.println(javaConcepts.info());  
  
        Dico petitRobert = new Dico("Petit Robert",  
                                     "Robert", 5000, 60000);  
        System.out.println(petitRobert.info());  
    }  
}
```

"Java Concepts" de Cay Horstmann

"Petit Robert" de Robert (60000 définitions)



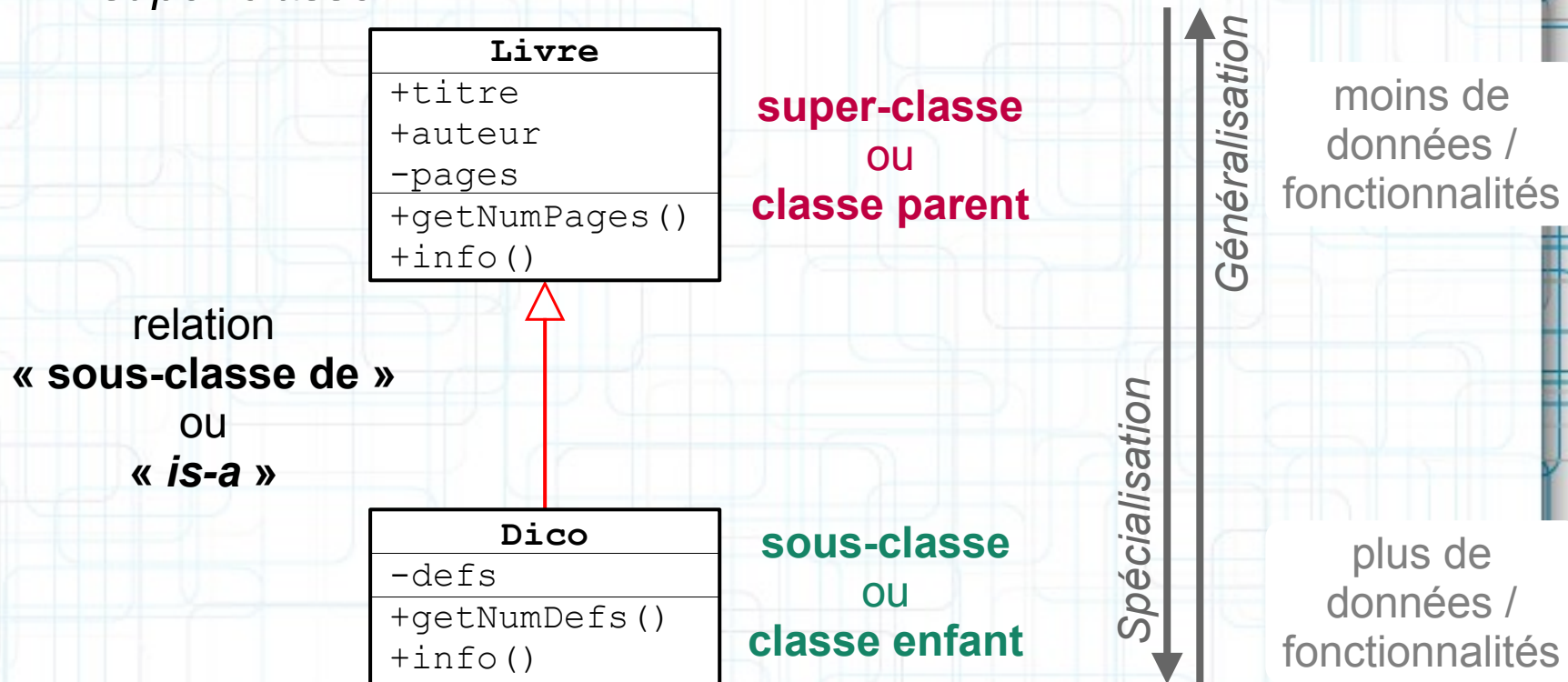
Table des Matières

- **Introduction**
 - Exemple introductif
 - Relation entre classes
- **Héritage en Java**
 - Déclaration de classe
 - Redéfinition de méthode
- **Classe Object**
- **Polymorphisme**
 - Transtypage
 - Liaison dynamique
- **Encapsulation revisitée**

Relation « is-a »

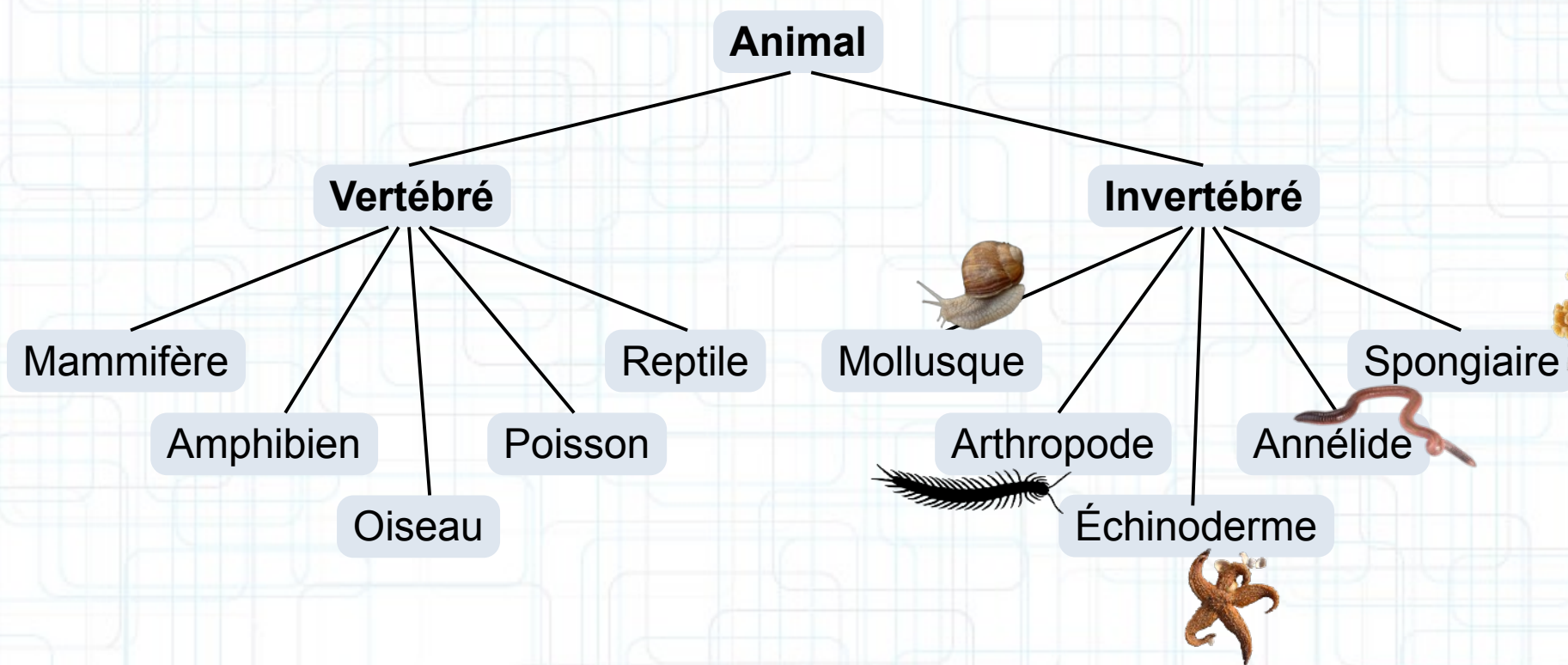
- **Relation entre classes**

- L'héritage permet que les données et le comportement d'une classe soient utilisés dans la définition d'une autre classe.
- Cela définit une **relation** entre classes dans laquelle une *sous-classe* partage la structure et le comportement définis par une *super-classe*.



Introduction

- **Analogie : classification des espèces**
 - La **relation** entre classes est souvent introduite par analogie avec la *classification des espèces*
 - p.ex. tout élément de la classe « *oiseau* » est aussi élément de la classe « *vertébré* »



Introduction

- Analogie : classification des espèces**

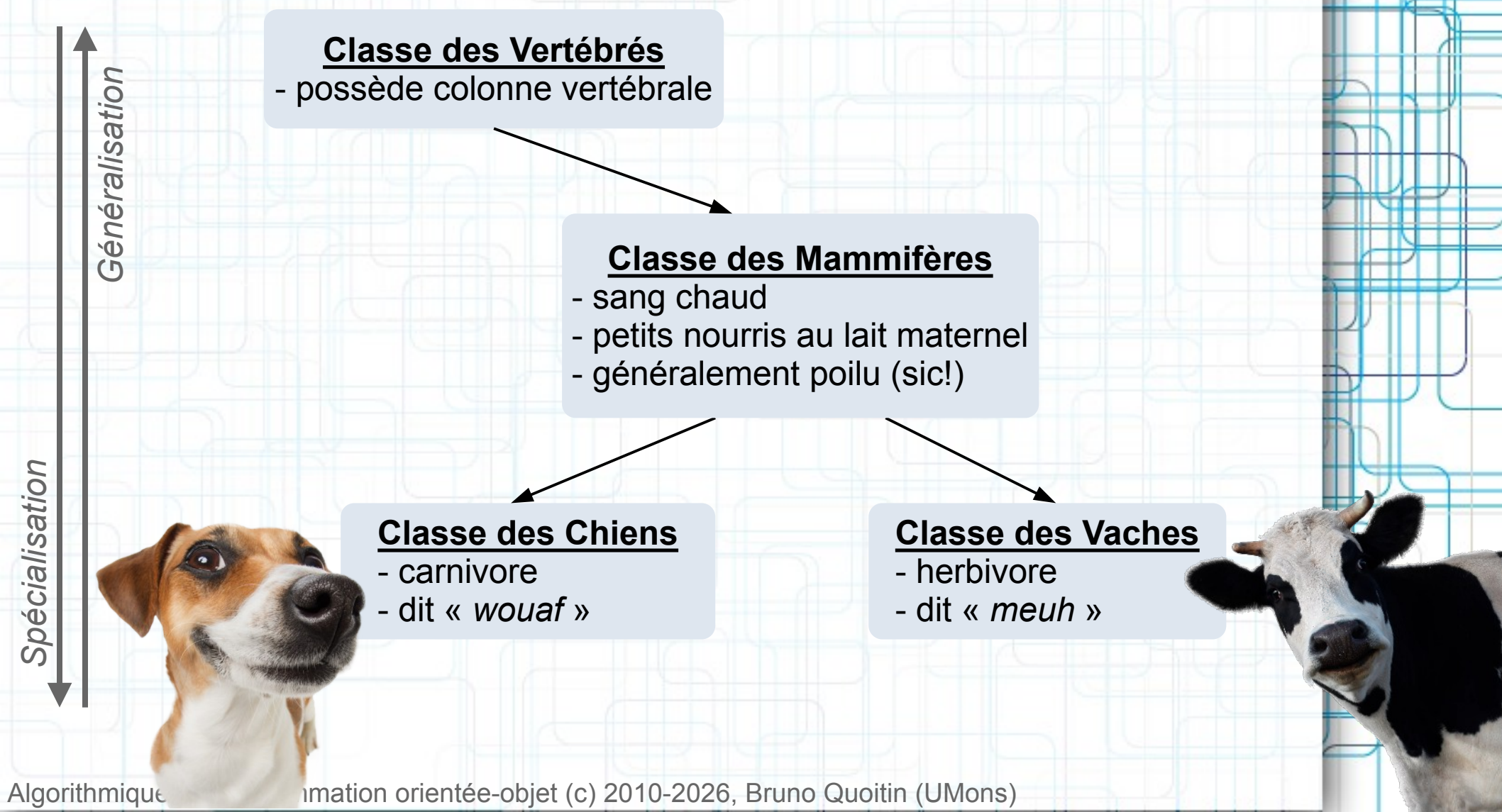


Table des Matières

- **Introduction**
 - Exemple introductif
 - Relation entre classes
- **Héritage en Java**
 - Déclaration de classe
 - Redéfinition de méthode
- **Classe Object**
- **Polymorphisme**
 - Transtypage
 - Liaison dynamique
- **Encapsulation revisitée**

Héritage

- **Héritage en Java**

- Le mot-clé **extends** permet d'indiquer qu'une classe est définie par héritage d'une autre classe.

```
public class Dico extends Livre { ... }
```

- Il s'agit d'un **héritage simple** : une classe ne peut avoir qu'un parent.
- Les classes qui ne spécifient pas de parent héritent implicitement de la **classe Object**. Par transitivité, toutes les classes héritent d'**Object**.
- Il est possible d'empêcher qu'une classe soit utilisée comme parent : lui ajouter le modificateur **final**.

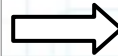
Héritage

- **Héritage en Java**

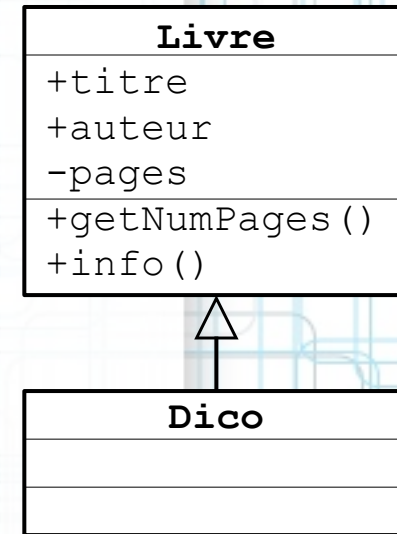
- L'héritage permet de ré-utiliser du code d'une autre classe.

```
public class Livre {
    public String titre;
    public String auteur;
    private int pages;
    public int getNumPages () {
        return pages;
    }
    public String info () {
        return "\"" + titre + "\"";
    }
}
```

```
public class Dico extends Livre {
}
```



```
public class Dico extends Livre {
    public String titre;
    public String auteur;
    private int pages;
    public int getNumPages () {
        return pages;
    }
    public String info () {
        return "Livre \"" + titre + "\"";
    }
}
```

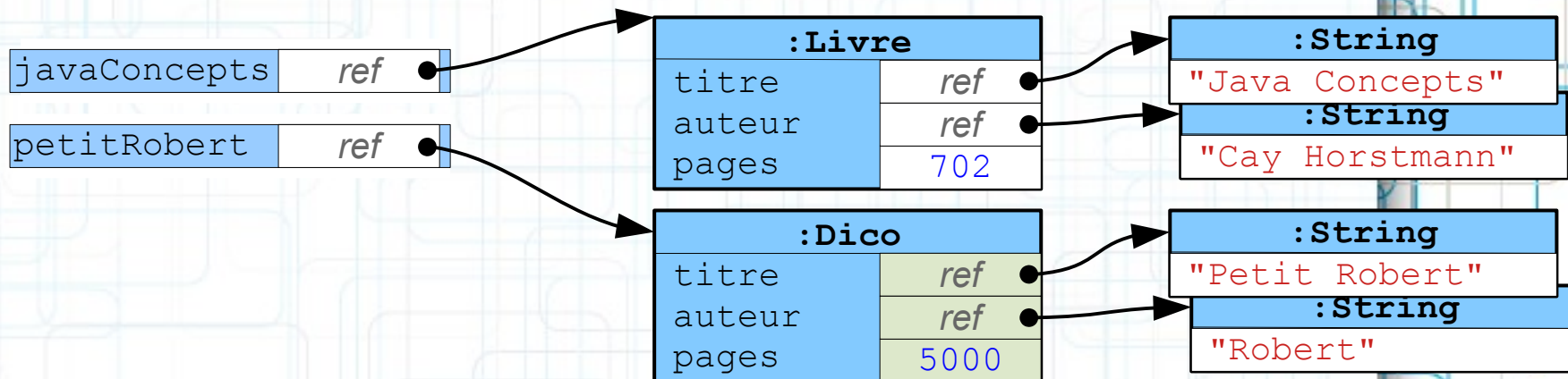
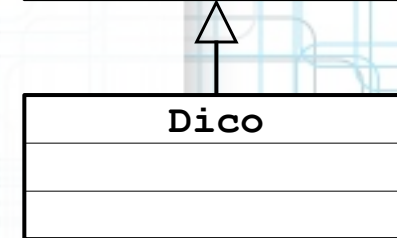
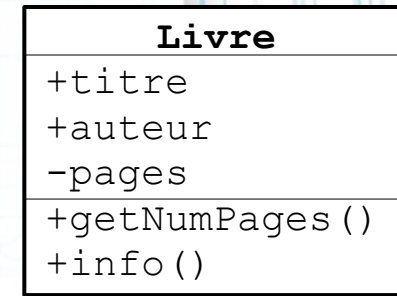


Héritage

- Héritage des membres**

- Tout les membres hérités de la classe parent peuvent être accédés à partir de la classe enfant comme s'ils y avaient été définis directement.

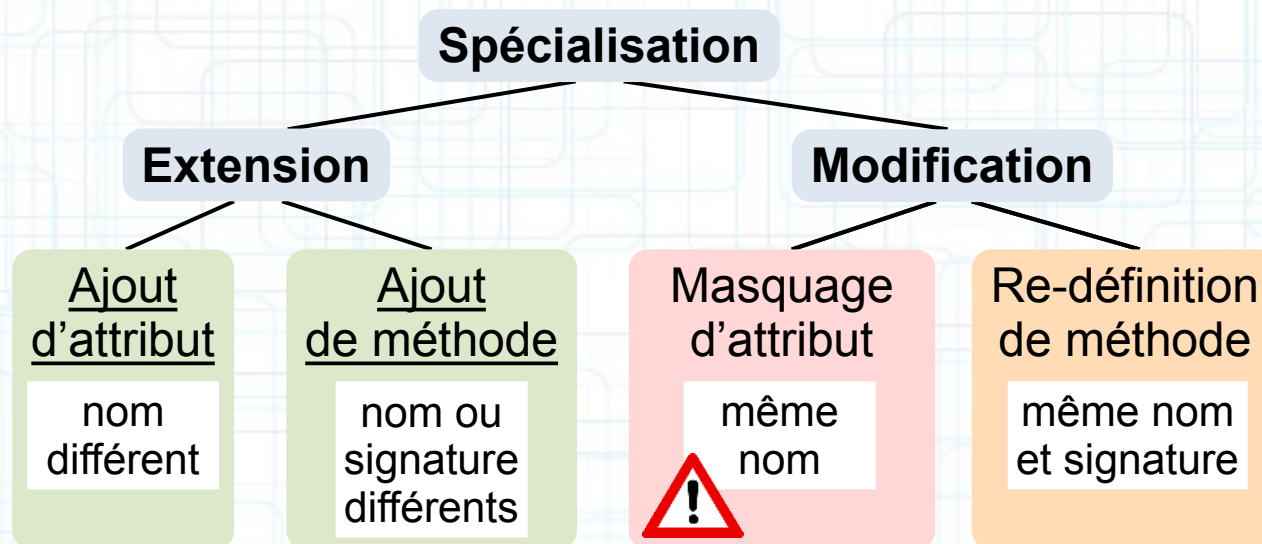
```
Livre javaConcepts = new Livre();
Dico petitRobert = new Dico();
System.out.println(petitRobert.titre);
System.out.println(petitRobert.getNumPages());
```



Héritage

- **Spécialisation**

- Une classe obtenue par héritage peut être **spécialisée** en adaptant la structure et le comportement par rapport à la super-classe.
- Cela peut être fait en ajoutant ou modifiant des attributs et méthodes.



Spécialisation

- **Ajout d'attributs et méthodes**
 - Attribut non existant (nom différent)
 - Méthode non existante (nom ou signature différents)

```
public class Livre {
    public String titre;
    public String auteur;
    private int pages;
    public int getNumPages() {
        return pages;
    }
    public String info() {
        return "\"" + titre + "\"";
    }
}
```

```
public class Dico extends Livre {
    private int defs;
    public int getNumDefs() {
        return defs;
    }
}
```

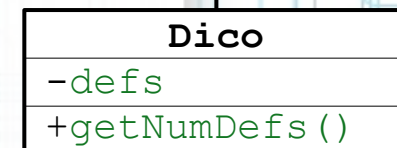
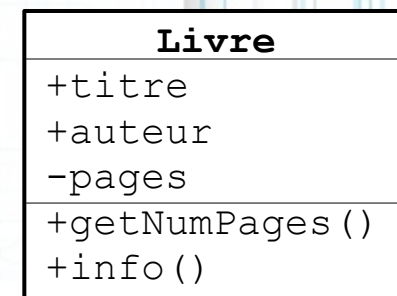


Table des Matières

- **Introduction**
 - Exemple introductif
 - Relation entre classes
- **Héritage en Java**
 - Déclaration de classe
 - Redéfinition de méthode
- **Classe Object**
- **Polymorphisme**
 - Transtypage
 - Liaison dynamique
- **Encapsulation revisitée**

Redéfinition de méthode

- **Re-définition de méthode**
 - Méthode héritée du parent doit être adaptée.
 - Nom et signature identiques à ceux de la méthode parent
 - re-définition = « *override* » versus surcharge = « *overload* »

Redéfinition de méthode

- Re-définition de méthode

```
public class Livre {
    public String titre;
    public String auteur;
    public String info() {
        return "\"" + titre + "\" de " + auteur;
    }
}
```

```
public class Dico extends Livre {
    private int defs;
    public String info() {
        return "\"" + titre + "\" de " + auteur +
            " (" + defs + " définitions)";
    }
}
```

```
Dico robert = new Dico("Petit Robert", "Robert",
                        5000, 60000);
System.out.println(dico.info());
```

```
"Petit Robert" de Robert (60000 définitions)
```

Livre
+titre
+auteur
-pages
+getNumPages()
+info()

Dico
-defs
+getNumDefs()
+info()



même nom / signature

Redéfinition de méthode

- Re-définition de méthode

```
public class Livre {
    public String titre;
    public String auteur;
    public String info() {
        return "\"" + titre + "\" de " + auteur;
    }
}
```

même nom / signature

```
public class Dico extends Livre {
    private int defs;
    public String info() {
        return info() + " (" + defs + " définitions)";
        return super.info() + " (" + defs + " définitions)";
    }
}
```

 Appels récursifs infinis !

Appel à la méthode du parent

```
Dico robert = new Dico("Petit Robert", "Robert",
                        5000, 60000);
System.out.println(dico.info());
```

"Petit Robert" de Robert (300000 définitions)

Livre
+titre
+auteur
-pages
+getNumPages()
+info()

Dico
-defs
+getNumDefs()
+info()



Redéfinition de méthode

- **Annotation @Override**

- Lors d'une redéfinition de méthode, les erreurs suivantes peuvent se produire
 - la méthode à redéfinir n'existe pas dans un parent
 - la méthode est mal-orthographiée ou sa signature n'est pas respectée
- Afin d'aider le compilateur à détecter ces erreurs, il est possible (et recommandé) d'utiliser l'annotation **@Override** afin de manifester son intention.
- Exemple

```
public class Dico extends Livre {  
    @Override  
    public String ingfo() {  
        return super.info() + " (" + defs + " définitions)";  
    }  
}
```

Le compilateur peut détecter qu'il n'existe pas de méthode **ingfo()** dans le parent (Livre).

Redéfinition de méthode

- **Re-définition de méthode**

- Détails techniques

- Une méthode définie avec le modificateur **final** ne peut être re-définie.
 - Les méthodes **private** ne sont pas visibles par une sous-classe. Elles ne peuvent donc pas être ré-définies; elles sont donc **final**.
 - La re-définition d'une méthode peut élargir l'accès mais pas le rétrécir. Par exemple, **protected** → **public** est autorisé
 - La re-définition d'une méthode qui retourne un objet peut avoir un type de retour différent mais pas plus large (voir références polymorphiques plus tard).

Spécialisation

- **Ajout d'un constructeur**

- Il est possible de re-définir un constructeur dans la sous-classe.
- Cependant, si la classe parent a un constructeur qui prend des arguments, il est **obligatoire** d'appeler ce constructeur en utilisant **super (...)** à la 1^{ère} ligne.

```
public class Livre {  
    public final String titre;  
    public final String auteur;  
    public Livre(String titre, String auteur) {  
        this.titre = titre;  
        this.auteur = auteur;  
    }  
}
```

```
public class Dico extends Livre {  
    public Dico(String titre, String auteur) {  
        super(titre, auteur); -----> à la 1ère ligne !  
    }  
}
```

Spécialisation

- **Redéfinition de variable**

- Peut-on redéfinir des variables ? **NON**
- Lorsqu'une variable d'une sous-classe masque une variable de la classe parent, en fait, les deux variables vont co-exister dans les instances de la sous-classe, mais seule une des deux pourra être accédée.

```
public class Maman {  
    public int x = 10;  
}
```

```
public class Enfant extends Maman {  
    public int x = 5;  
}
```



Héritage

- **Exercice**
 - Modélisation de comptes bancaires
 - Partie commune
 - structure : numéro de compte, propriétaire, solde
 - opérations : obtenir le solde, effectuer un dépôt, effectuer un retrait, appliquer un traitement mensuel (p.ex. calcul des intérêts)
 - Comptes épargnes
 - taux d'intérêt, appliqué mensuellement sur le solde minimum du mois
 - Comptes courants
 - opérations de retrait payantes (N gratuites par mois)

Héritage

- **Exercice**
 - Modélisation de comptes bancaires

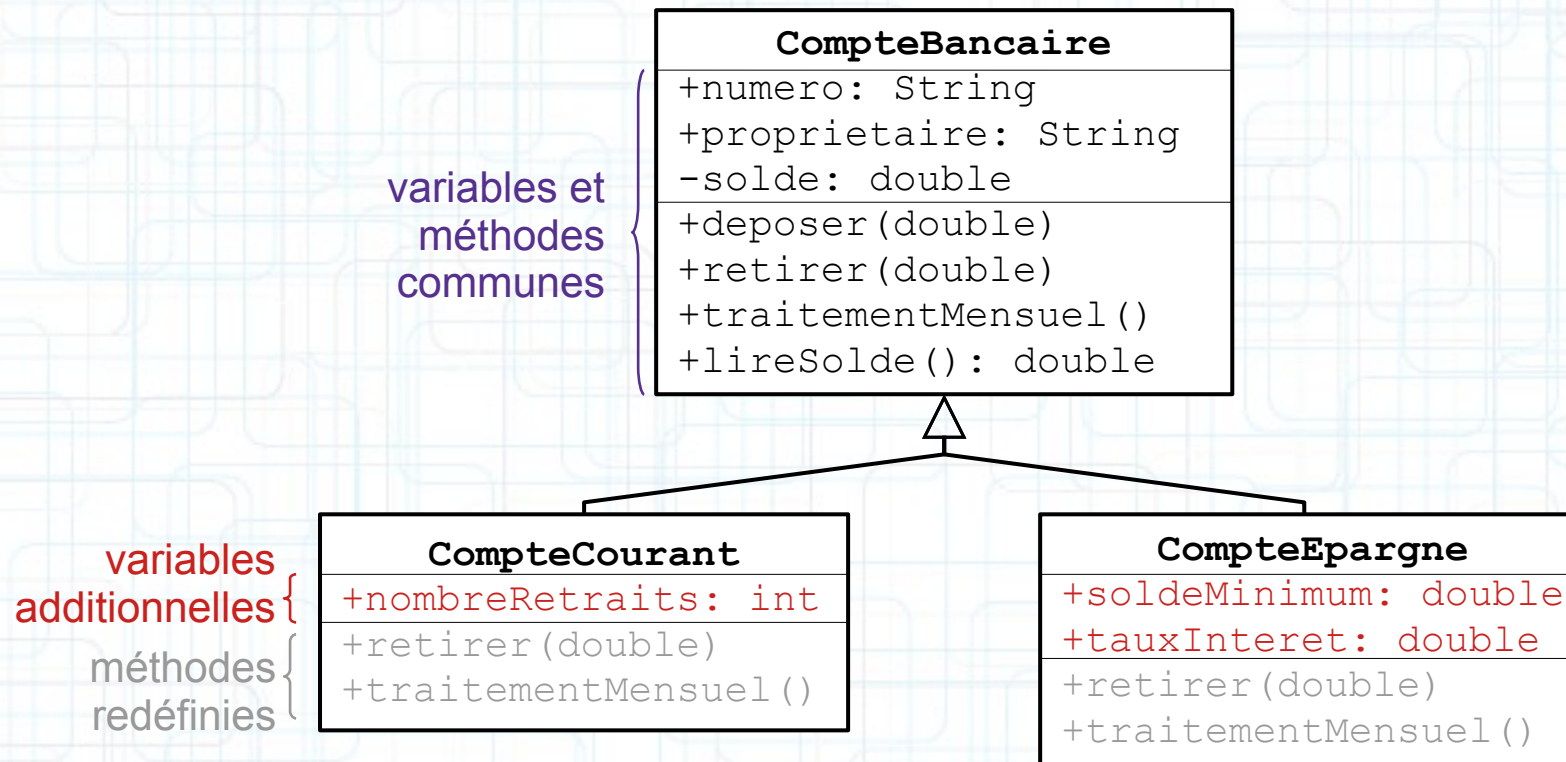


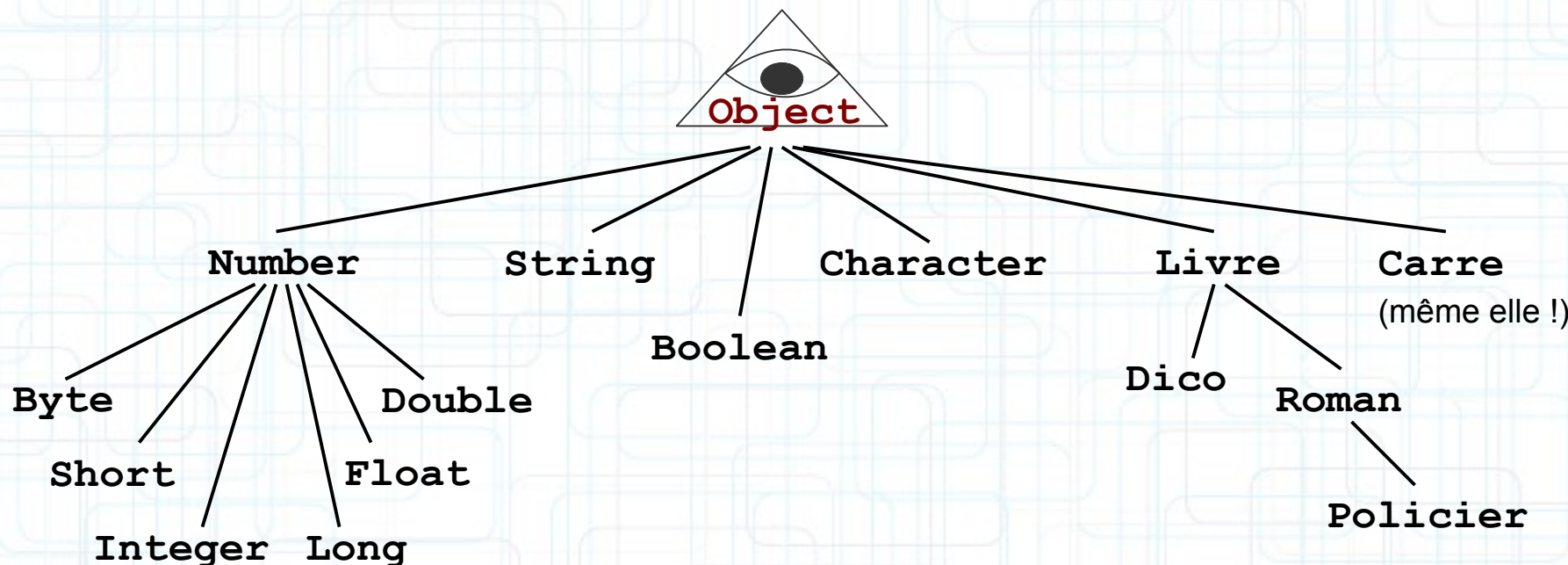
Table des Matières

- **Introduction**
 - Exemple introductif
 - Relation entre classes
- **Héritage en Java**
 - Déclaration de classe
 - Redéfinition de méthode
- **Classe Object**
- **Polymorphisme**
 - Transtypage
 - Liaison dynamique
- **Encapsulation revisitée**

Classe Object

- **La classe Object**

- En Java, toutes les classes descendent de la classe `Object`. (même sans utilisation du mot-clé **extends**).
- La classe `Object` est à la racine de toute la hiérarchie de classes. Elle n'a pas de parent.



Classe Object

- **Services fournis**

- La classe `Object` fournit un certain nombre de services qui sont donc communs à toutes les classes !

Méthode		Description
public	boolean <code>equals (Object)</code>	Test d'égalité
public	<code>String toString ()</code>	Représentation
public	int <code>hashCode ()</code>	Hachage
public final	<code>Class<?> getClass ()</code>	Classe d'une instance
protected	<code>Object clone ()</code>	Clone une instance
(...)		

Classe Object

- **Méthode equals ()**

- L'opérateur `==` teste l'égalité de deux valeurs. Pour des objets, il effectue donc un **test d'identité** : deux références ne sont égales que si elles mènent au même objet.
- La méthode `equals ()` rend possible le **test d'égalité** de deux objets sur base de leur contenu.
- Par défaut elle effectue un test d'égalité. Il est nécessaire de la re-définir dans les sous-classes.

```
public boolean equals (Object o) {  
    return this == o;  
}
```

Classe Object

- Méthode `equals()`

```
public class Livre {  
    public final String titre;  
    public final String auteur;  
    /* ... */  
    @Override  
    public boolean equals(Object o) {  
        if (o == null)  
            return false;  
        if (!(o instanceof Livre))  
            return false;  
        Livre ol = (Livre) o;  
        return titre.equals(ol.titre) && auteur.equals(ol.auteur);  
    }  
}
```

`x.equals(null) == false`

on ne peut être égal qu'à un objet de « même type »

Classe Object

- **Méthode toString()**

- Permet de fournir une **représentation** d'un objet sous forme d'une chaîne de caractères.
- Cette représentation est surtout utilisée pour du débogage.; elle est typiquement la plus brève possible.
- Le comportement par défaut de `toString()` ressemble à ceci

```
public String toString() {  
    return getClass().getName() + "@" +  
           Integer.toHexString(hashCode());  
}
```

Livre@659e0bfd

- Il est recommandé de la re-définir dans les sous-classes.

Classe Object

- Méthode toString()

```
public class Livre {  
    /* ... */  
    @Override  
    public String toString() {  
        return getClass().getName() + "\" + titre +  
            "\" écrit par \" + auteur;  
    }  
}
```

→ Pourquoi ?

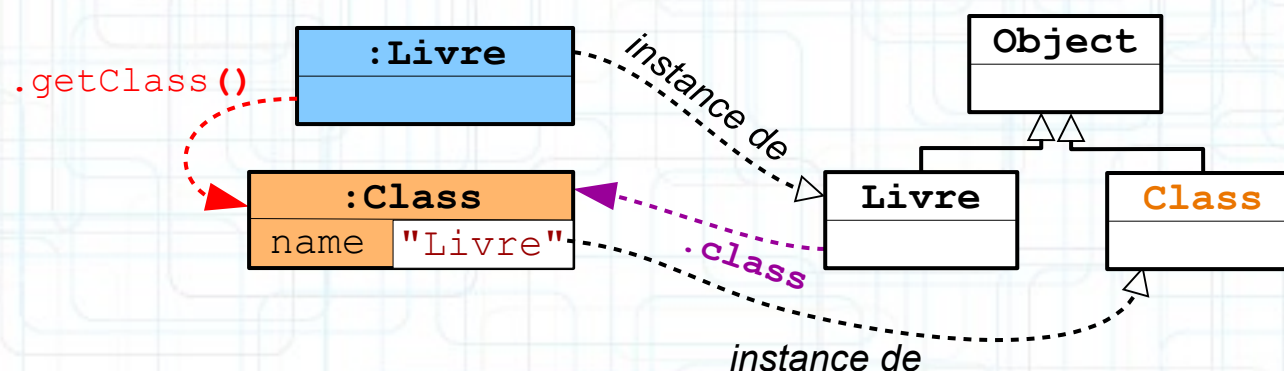
```
public class Dico extends Livre {  
    /* ... */  
    @Override  
    public String toString() {  
        return super().toString() + " (" + defs + " définitions)";  
    }  
}
```

```
Livre "Java Concepts" écrit par Cay Horstmann  
Dico "Petit Robert" écrit par Robert (60000 définitions)
```

Classe Object

- **Méthode getClass()**

- Une partie de l'information de typage est rendue accessible durant l'exécution sous la forme d'instances de la **classe Class**. On parle de « *Run-time Type Identification* » (RTTI).
- A chaque classe est associée une **instance de Class**. Celle-ci offre des méthodes telles que `getName()`, `getFields()`, `getConstructors()`, `getMethods()`, ...



- La méthode `getClass()` retourne la classe d'une instance.
- Le littéral `.class` donne l'instance de `Class` associée à une classe (p.ex. `Livre.class`)

Classe Object

- **Méthode getClass()**

```
Livre javaConcepts = new Livre();  
Dico petitRobert = new Dico();  
System.out.println(javaConcepts.getClass().getName());  
System.out.println(petitRobert.getClass().getName());  
System.out.println(Arrays.toString(Dico.class.getFields()));
```

```
Livre  
Dico  
[public final int Dico.defs, public final java.lang.String  
Livre.auteur, public final java.lang.String Livre.titre,  
public final int Livre.pages]
```

Classe Object

- Classes et instances de Class

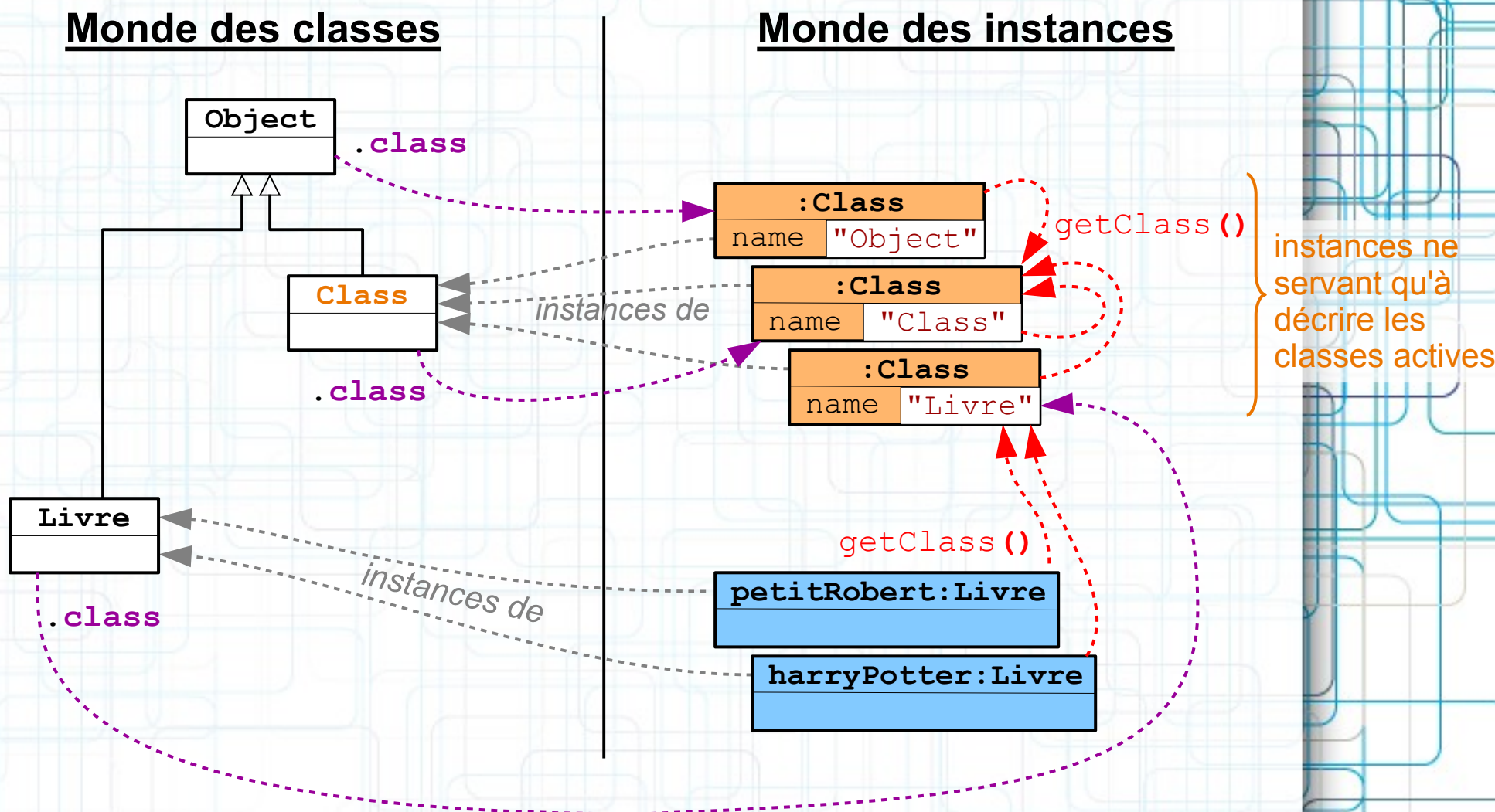


Table des Matières

- **Introduction**
 - Exemple introductif
 - Relation entre classes
- **Héritage en Java**
 - Déclaration de classe
 - Redéfinition de méthode
- **Classe Object**
- **Polymorphisme**
 - Transtypage
 - Liaison dynamique
- **Encapsulation revisitée**

Polymorphisme

- **Polymorphisme**

- Capacité à **manipuler des objets de types/formes différents comme s'ils étaient d'un même type**. Probablement l'une des fonctionnalités les plus importantes de la P.O.O. !!!
- Héritage = un des mécanismes qui rendent le polymorphisme possible
 - Lorsque deux classes sont en relation d'héritage, ce que nous notons : $A \triangleright B$,
 - toute instance de la sous-classe A peut-être manipulée comme si elle était instance de la super-classe B .
 - Cette forme est appelée **polymorphisme d'inclusion**.

Polymorphisme

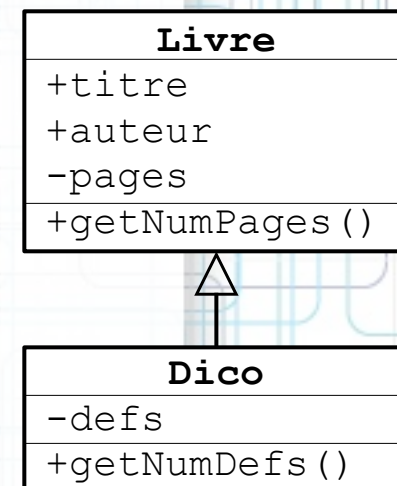
- **Polymorphisme d'inclusion**

- Une instance d'une sous-classe (p.ex. `Dico`) peut être stockée dans une variable dont le type est la super-classe (p.ex. `Livre`)
 - On peut accéder à tout attribut / méthode défini dans la super-classe.

Type déclaré

Type réel

```
Livre petitRobert = new Dico("Petit Robert",
                             "Robert", 5000, 60000);
System.out.println(petitRobert.titre);
System.out.println(petitRobert.getNumPages());
```



- Une telle référence est appelée **référence polymorphique**.
- Le type déclaré **D** et le type réel **R** peuvent être différents, cependant il est nécessaire que **R** $\triangleright$ **D**
- **Le compilateur fait respecter cette contrainte !**

Polymorphisme

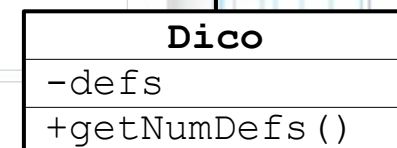
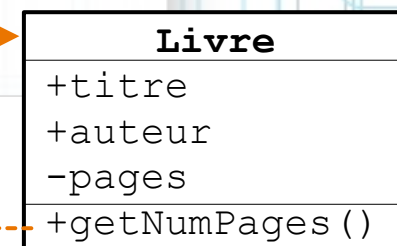
- **Polymorphisme d'inclusion**

- Extrêmement puissant avec des collections d'objects (p.ex. tableaux)

```
public static double avgNumPages(Livre[] livres) {
    int total = 0;
    for (Livre l: livres)
        total += l.getNumPages();
    return (1.0 * total) / livres.length;
}
```

```
Livre[] livres = {
    new Livre("Le trône de fer", "G.R.R. Martin", 477),
    new Livre("Harry Potter à l'école des sorciers",
        "J.K. Rowling", 312),
    new Dico("Petit Larousse illustré", "Larousse", 2000,
        63500),
    new Livre("The Nature of Code", "D. Shiffman", 520),
    new Dico("Trump Dictionary", "F. Isaacs", 1, 15),
};
System.out.println(avgNumPages(livres)); /* 662.0 */
```

Type déclaré



Polymorphisme

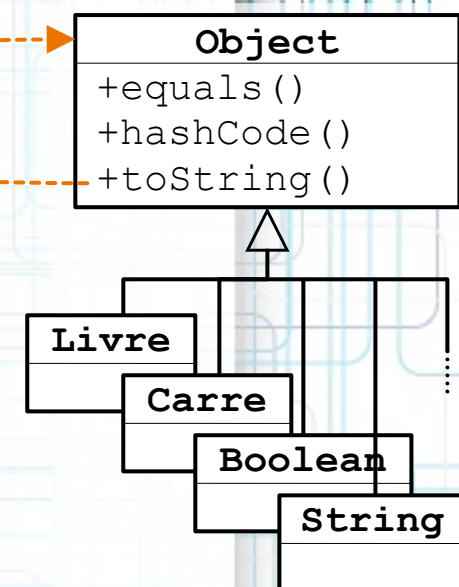
- **Polymorphisme d'inclusion**

- La méthode `println()` de la classe `PrintStream` utilise cette forme de polymorphisme pour afficher la représentation de n'importe quel objet.

```
public void println(Object o) {  
    if (o == null)  
        print("null");  
    else  
        print(o.toString());  
    println();  
}
```

Type déclaré

```
System.out.println(new Livre("L'étranger",  
                             "A. Camus", 185));  
System.out.println(new Carre(4, 7, 5, -60));  
System.out.println(Boolean.FALSE);  
System.out.println("Ceci n'est pas une chaîne");  
/* ... */
```



Polymorphisme

- **Relation d'ordre**

- La relation « *is-a* » ou « *est une sous-classe de* », que nous avons notée « $\triangleright$ », est une **relation d'ordre partiel** sur l'ensemble des classes car elle satisfait les propriétés suivantes :

- **Antisymétrique**

$$\forall A, B : (A \triangleright B) \text{ et } (B \triangleright A) \Rightarrow A = B$$

- **Transitive**

$$\forall A, B, C : (B \triangleright A) \text{ et } (C \triangleright B) \Rightarrow C \triangleright A$$

- **Réflexive**

$$\forall A : A \triangleright A$$

- Corollaire pour le polymorphisme : le type (réel) d'un objet référencé n'est pas nécessairement une sous-classe directe du type déclaré.

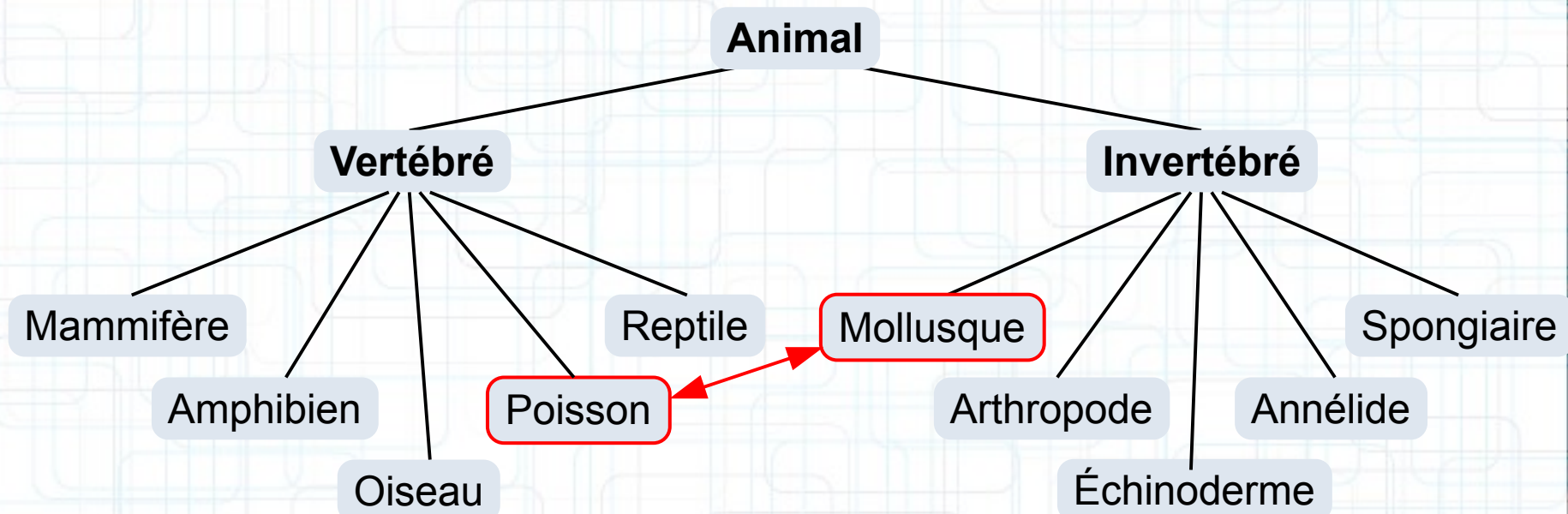
Polymorphisme

- **Relation d'ordre**

- Il ne s'agit pas d'une relation d'ordre total. Il faudrait en effet pour cela que toutes les classes soient en relation **is-a**, c-à-d.

$$\forall A, B : (A \triangleright B) \text{ ou } (B \triangleright A)$$

- L'illustration ci-dessous, tirée de l'analogie avec la classification des espèces, donne un contre-exemple.



un Poisson **n'est pas** un Mollusque
un Mollusque **n'est pas** un Poisson

Polymorphisme

• Tester la classe d'une instance ?

- Avec les références polymorphiques, le type déclaré et le type réel peuvent être différents.
- Comment tester si un objet est un sous-type d'une certaine classe ? Opérateur **instanceof**

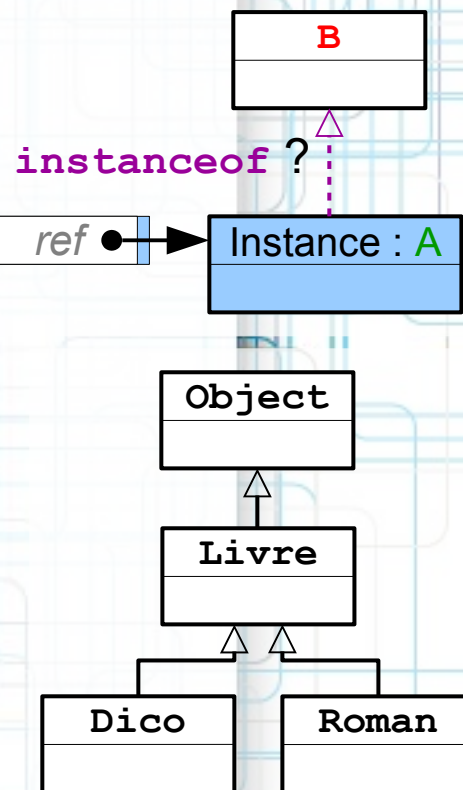
référence instanceof nomClasse

- premier argument : référence vers un objet
- deuxième argument : nom de classe
- Retourne **true** $\Leftrightarrow$ **type réel** $\triangleright$ **classe**

- Exemple

```
Object ref = new Dico();
System.out.println(ref instanceof Dico);
System.out.println(ref instanceof Livre);
System.out.println(ref instanceof String);
```

true
true
false



Type
déclaré

Type réel

Polymorphisme

• Tester la classe d'une instance ?

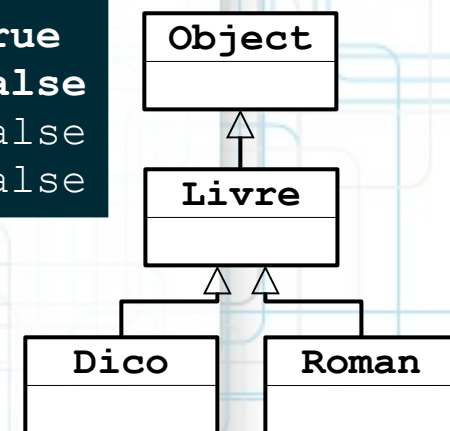
- L'opérateur **instanceof** permet de tester si un objet est issu d'une classe en relation d'héritage avec une certaine classe cible. Il ne permet pas de tester si l'objet est issu directement de cette classe.
- Il est possible d'utiliser pour cela la méthode `getClass()` de la classe `Object`.

Type réel

Type déclaré

```
Object ref = new Dico();
System.out.println(ref instanceof Dico);
System.out.println(ref.getClass() == Dico.class);
System.out.println(ref instanceof Livre);
System.out.println(ref.getClass() == Livre.class);
System.out.println(ref instanceof String);
System.out.println(ref.getClass() == String.class);
```

true
true
true
false
false
false



Polymorphisme

- **Opérateur `instanceof`**
 - **Attention !** L'opérateur `instanceof` est réservé à des cas bien particuliers.
 - Normalement, dans vos programmes il ne devrait pas être nécessaire de l'utiliser.
 - Si vous êtes amenés à utiliser `instanceof`, c'est peut-être parce que votre approche n'est pas orientée-objet et qu'elle doit être retravaillée...

Polymorphisme

- **Autres formes de polymorphisme**

- On distingue 4 types de polymorphismes [Cardelli et al, 1985]

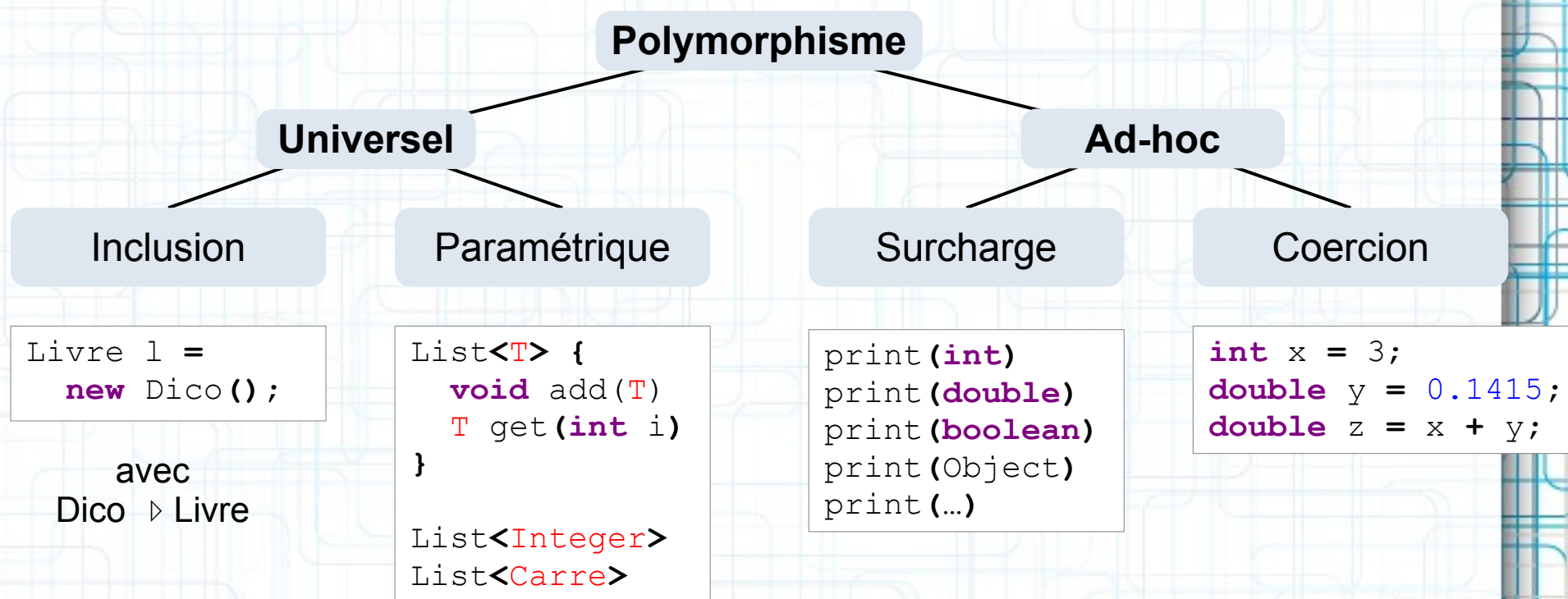


Table des Matières

- **Introduction**
 - Exemple introductif
 - Relation entre classes
- **Héritage en Java**
 - Déclaration de classe
 - Redéfinition de méthode
- **Classe Object**
- **Polymorphisme**
 - Transtypage
 - Liaison dynamique
- **Encapsulation revisitée**

Transtypage

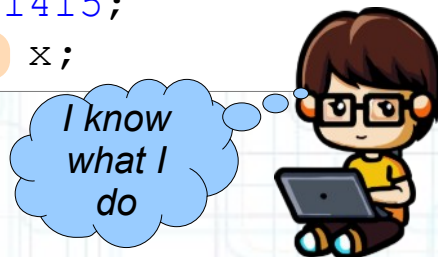
- **Conversion de types**

- Le **transtypage** (« *typecasting* ») est le mécanisme permettant de changer le type d'une entité. Par exemple on pourrait vouloir convertir un **int** en **double** :

```
int x = 5;  
double y = x;
```

- Un transtypage peut être
 - **implicite** : le compilateur sait vérifier que la conversion est valide
 - **explicite** : le compilateur ne sait pas vérifier la validité (manque d'information) ⇒ il faut demander la conversion avec un **opérateur de transtypage**.

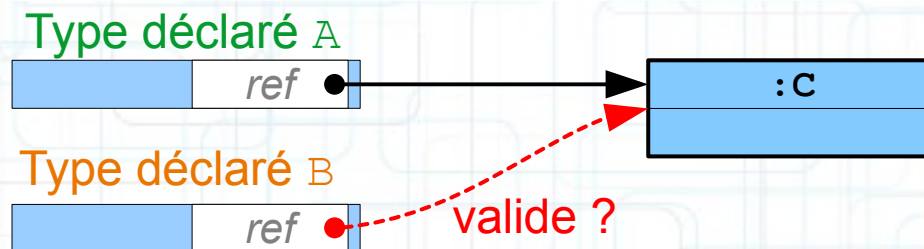
```
double x = 3.1415;  
int y = (int) x;
```



Transtypage

- **Conversion de types objet**

- On s'intéresse au **transtypage objet**, c-à-d au changement du type d'une référence.
- Le cadre général est le suivant : on veut convertir une référence de **type A** vers une référence de **type B**. Le type réel de l'objet référencé est C.
 - le compilateur connaît les types déclarés **A** et **B**
 - mais pas le type réel C



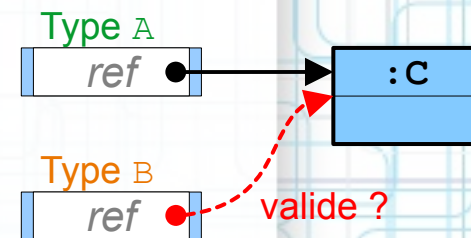
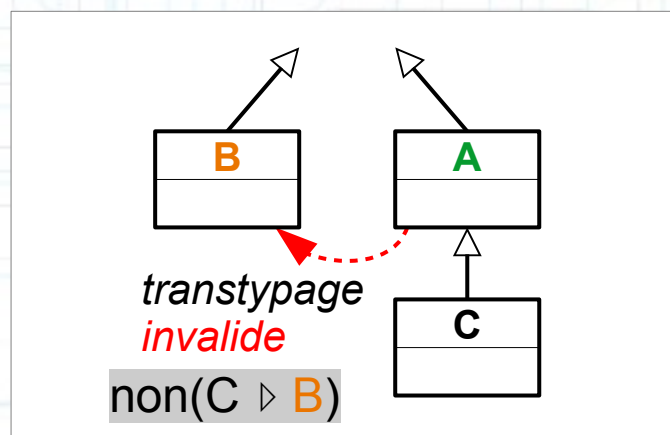
- Le transtypage est valide si $C \triangleright B$

Transtypage

- Trois cas à couvrir

- Cas 1 - Transtypage invalide

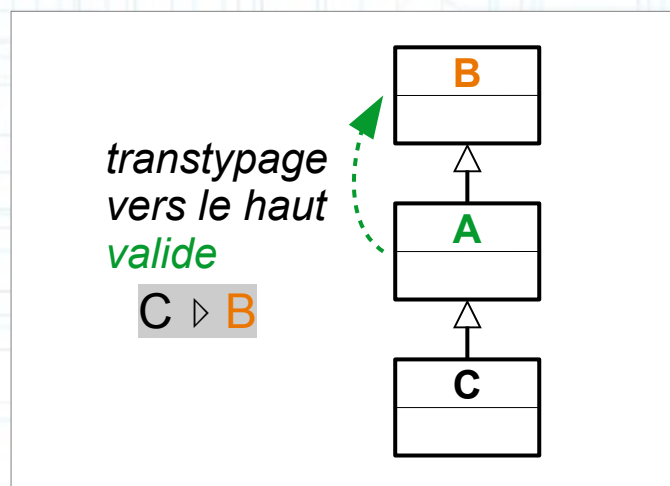
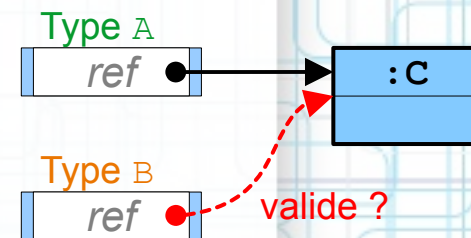
- Cas : les types déclarés A et B ne sont pas liés par l'héritage, autrement dit $\text{non}(A \triangleright B)$ et $\text{non}(B \triangleright A)$
- Transtypage invalide $\Rightarrow$ **refusé** par le compilateur



Transtypage

• Cas 2 - Transtypage vers le haut (*upcasting*)

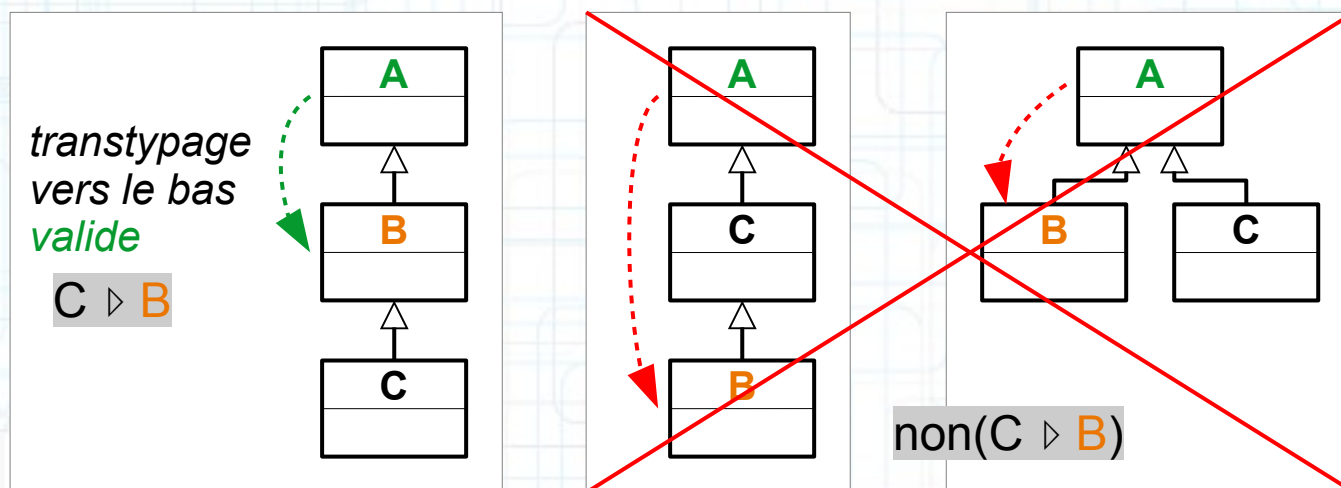
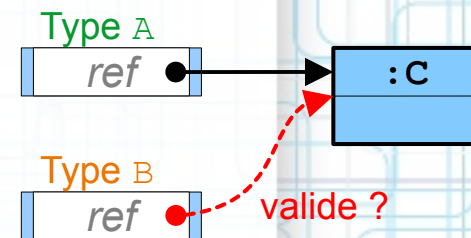
- Cas : le type déclaré d'origine, **A**, est un sous-type du type déclaré destination, **B**, autrement dit $A \triangleright B$
- Par pré-condition, on sait que $C \triangleright A$. On déduit par transitivité que $C \triangleright B$
- Conversion *implicite* **acceptée** par le compilateur



Transtypage

• Cas 3 - Transtypage vers le bas (*downcasting*)

- Cas : le type déclaré d'origine, **A**, est un super-type du type déclaré destination, **B**, autrement dit $B \triangleright A$
- Conversion implicite **refusée** par le compilateur. Le compilateur ne peut rien conclure car plusieurs situations sont possibles : certaines valides, d'autres pas.
- Conversion explicite **acceptée** mais vérification par la JVM sur base du type réel **C** de l'instance. Si échec, déclenche `ClassCastException`.



Transtypage

- Quels transtypes sont valides ?

```
Roman r = new Dico();
```

invalide !

```
Livre l = new Policier();
```

upcasting

```
Livre l = new Policier();  
Roman r = l;  
Roman r = (Roman) l;
```

downcasting implicite
downcasting explicite valide

```
Livre l = new Roman();  
Policier r = (Policier) l;
```

downcasting explicite invalide
→ `ClassCastException`

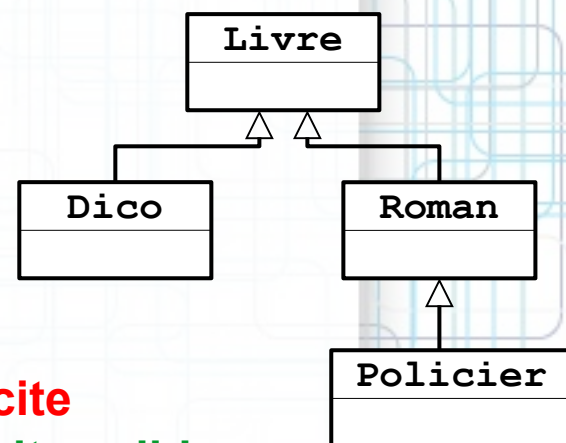


Table des Matières

- **Introduction**
 - Exemple introductif
 - Relation entre classes
- **Héritage en Java**
 - Déclaration de classe
 - Redéfinition de méthode
- **Classe Object**
- **Polymorphisme**
 - Transtypage
 - Liaison dynamique
- **Encapsulation revisitée**

Liaison dynamique

- **Introduction**

- La combinaison des références polymorphiques avec la ré-définition de méthodes permet à des objets de répondre différemment à l'invocation d'une méthode.
- Le mécanisme sous-jacent s'appelle **liaison dynamique** ou **liaison tardive**.

Note : *dynamic / late dispatch* ou *dynamic / late binding* sont les noms utilisés dans la littérature.

Liaison dynamique

- **Exemple**

- Trois classes $C \triangleright B \triangleright A$

```
public class A {
    public void showMessage() {
        System.out.println("Methode de A");
    }
}
```

```
public class B extends A {
}
```

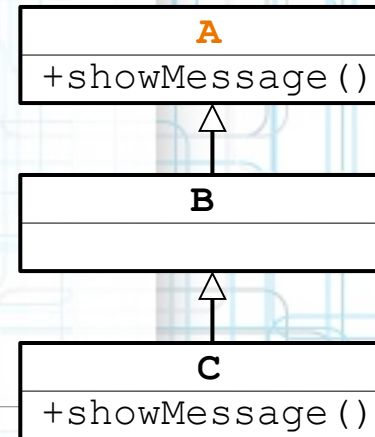
```
public class C extends B {
    public void showMessage() {
        System.out.println("Methode de C");
    }
}
```

```
A ref = new A();
ref.showMessage();
ref = new B();
ref.showMessage();
ref = new C();
ref.showMessage();
```

Methode de A

Methode de A

Methode de C

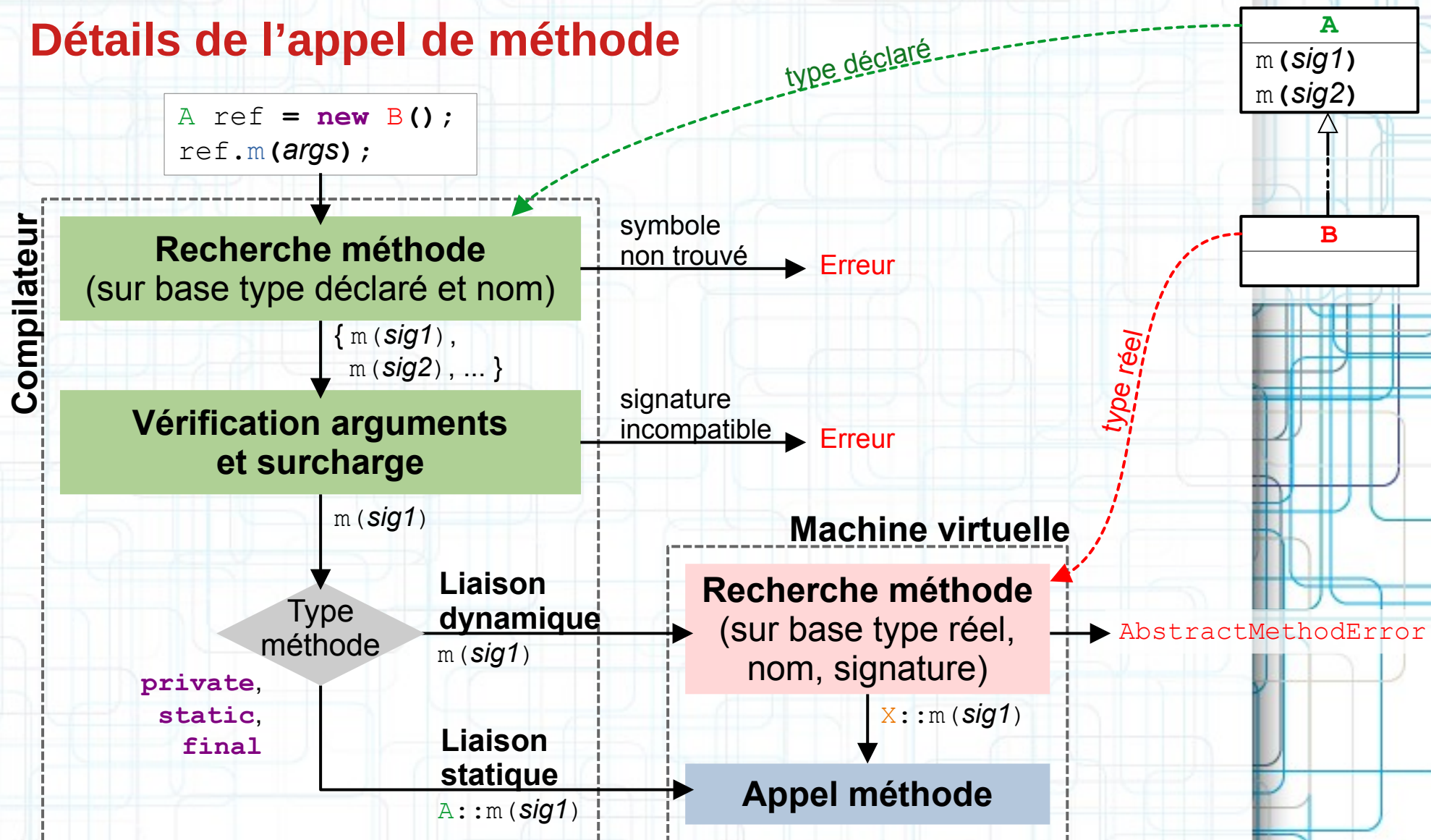


Le **type déclaré** est identique dans les 3 cas (A) → le compilateur ne fait pas la différence.

Le **type réel** est différent (A, B ou C) → la JVM détermine quelle méthode appeler, lors de l'exécution

Liaison dynamique

- Détails de l'appel de méthode



Liaison dynamique

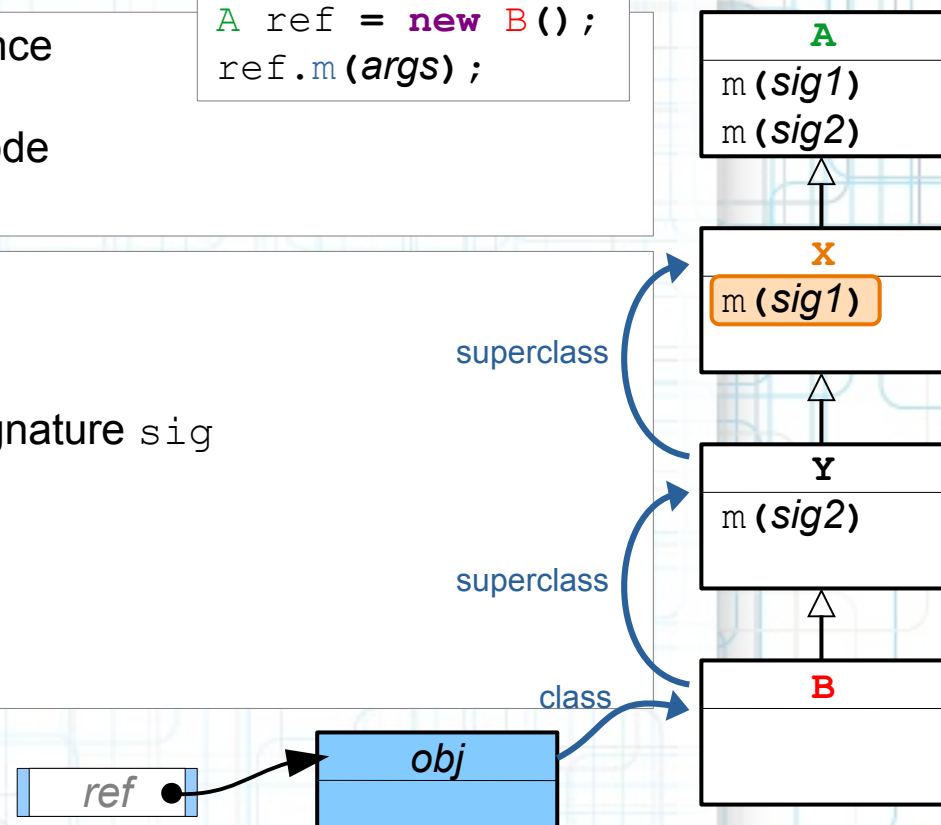
• Résolution de méthode par la JVM

- La machine virtuelle utilise l'algorithme suivant pour trouver (« résoudre ») une méthode d'instance à partir d'une référence et d'un nom de méthode (symbole).

entrée : **ref** est la référence vers l'instance
m est le nom de la méthode
sig est la signature de la méthode
sortie : méthode

```
A ref = new B();
ref.m(args);
```

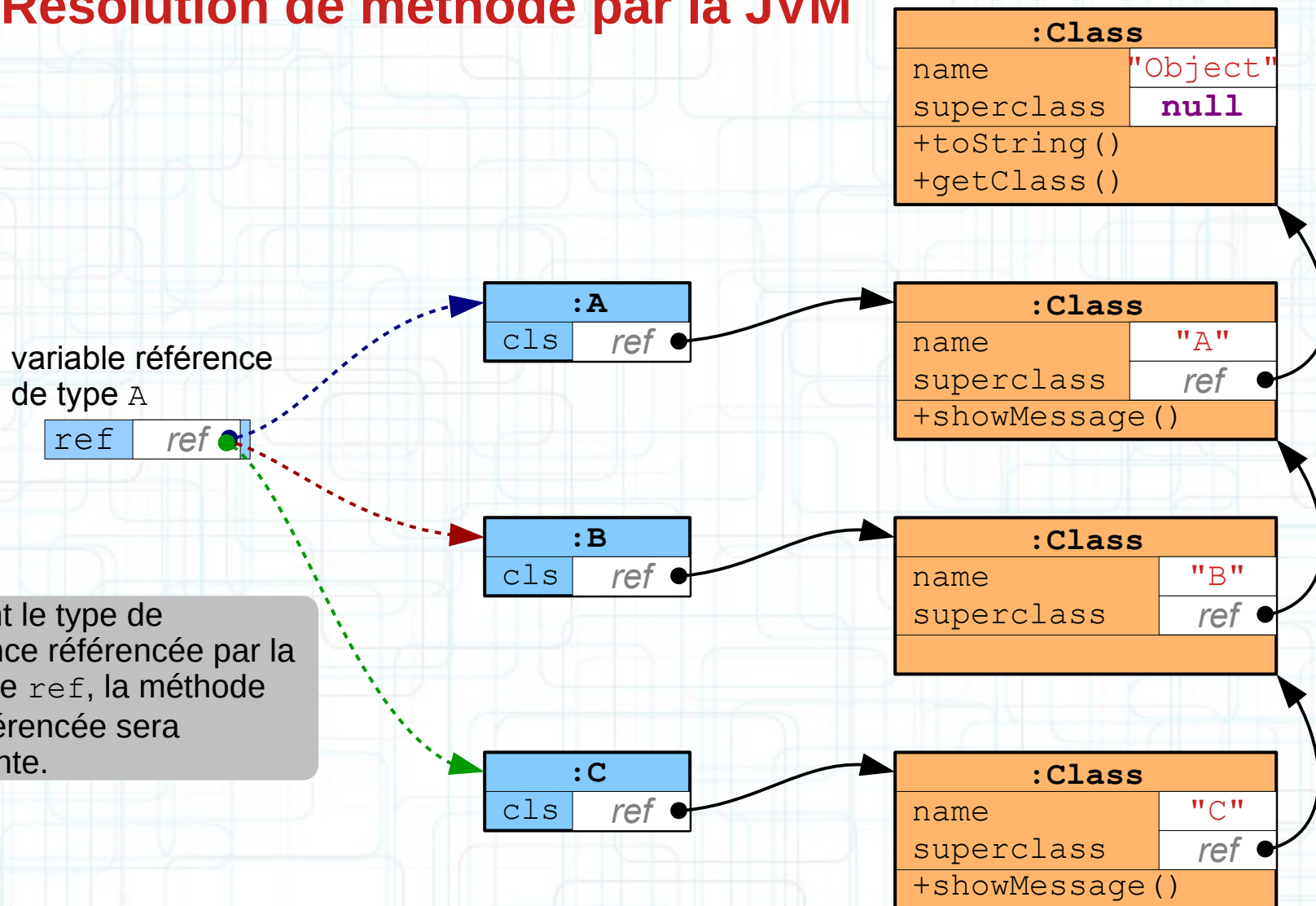
```
obj = object(ref)
cls = class(obj)
faire
  si cls déclare la méthode m() de signature sig
    retourner cls::m
  sinon
    cls = superclass(cls)
tant que (cls != null)
déclenche AbstractMethodError
```



Note : une description plus précise de cet algorithme est disponible dans la spécification de la machine virtuelle Java pour l'instruction `invokevirtual`.

Liaison dynamique

- Résolution de méthode par la JVM



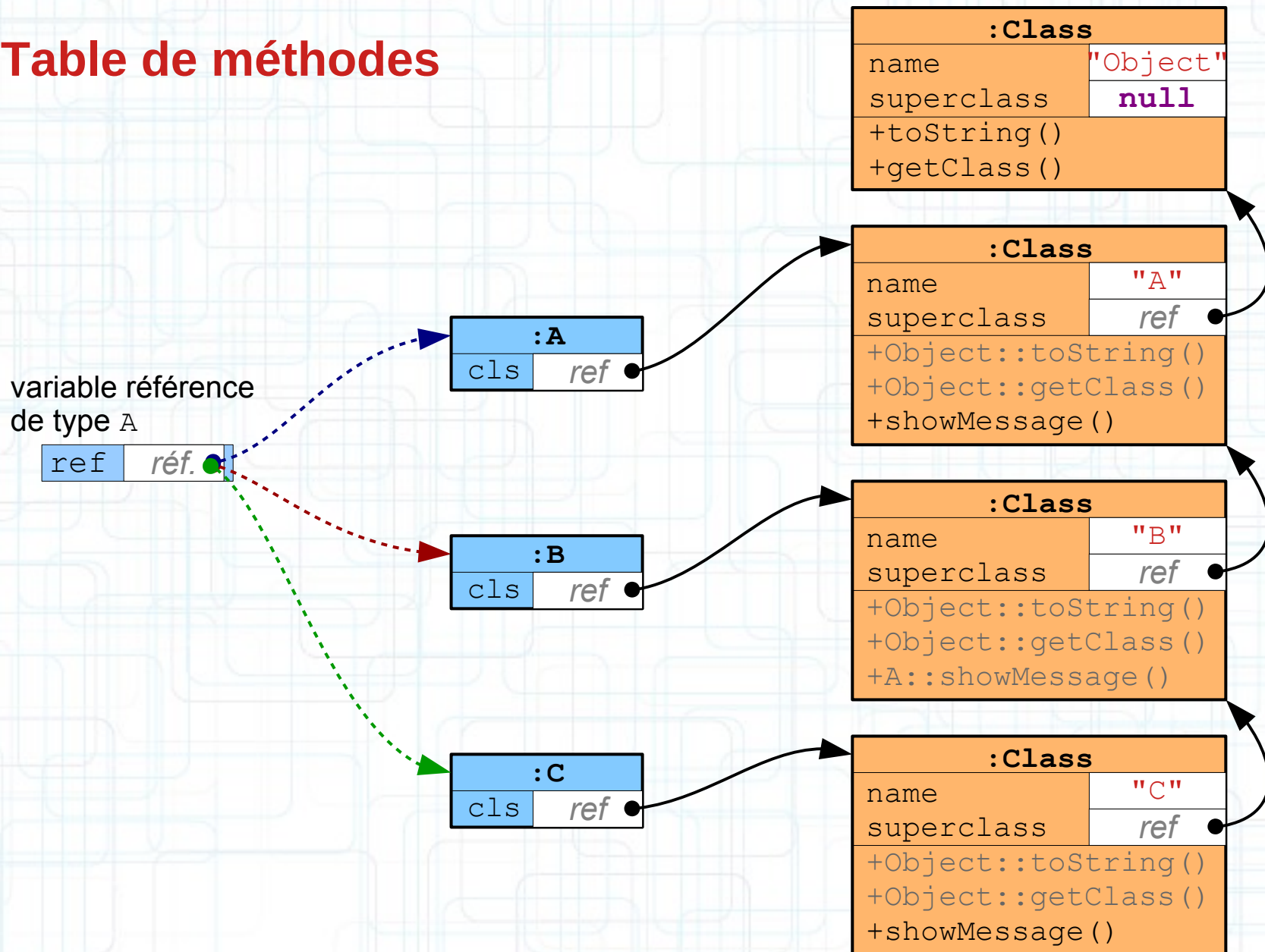
Liaison dynamique

- **Table de méthodes**

- Il serait trop lent d'exécuter l'algorithme précédent à chaque appel de méthode.
- En pratique, la machine virtuelle **pré-calcule** une **table de méthodes** pour chaque classe.
- Lorsqu'une méthode est appelée avec liaison dynamique, un simple accès à la table de la classe de l'objet permet d'obtenir la bonne méthode.

Liaison dynamique

- Table de méthodes



Liaison dynamique

- **Liaison statique pour les variables**

- Les variables d'instance et de classe sont accédées avec une liaison statique, c'est-à-dire uniquement sur base du type déclaré.
- Impact sur la « re-définition » d'attribut (masquage) ?

```
public class Maman {  
    public int x = 10;  
}
```

```
public class Enfant extends Maman {  
    public int x = 5;  
}
```

```
Enfant ref1 = new Enfant();  
Maman ref2 = ref1;  
System.out.println(ref1.x); /* affiche 5 */  
System.out.println(ref2.x); /* affiche 10 */
```



Liaison statique vs dynamique

• Exercice

```
public class Maman {
    public int x = 10;
    public void setX(int x) {
        this.x = x; }
    public int getX() {
        return x; }
}
```

```
public class Enfant extends Maman {
    public int x = 5;
    public void setX(int x) {
        this.x = x+1; }
}
```

```
Enfant enfant = new Enfant();
Maman maman = enfant;
System.out.println(enfant.x);
System.out.println(enfant.getX());
System.out.println(maman.x);
System.out.println(maman.getX());
enfant.setX(0);
System.out.println(enfant.x);
System.out.println(maman.x);
maman.setX(100);
System.out.println(enfant.x);
System.out.println(maman.x);
```

5
10
10
10

1
inchangé

101
inchangé

Early-binding → Enfant.x

Late-binding → Maman.getX(),
puis early-binding → Maman.x

Early-binding → Maman.x

Late-binding → Maman.getX(),
puis early-binding → Maman.x

Late-binding → Enfant.setX(),
puis early-binding → Enfant.x

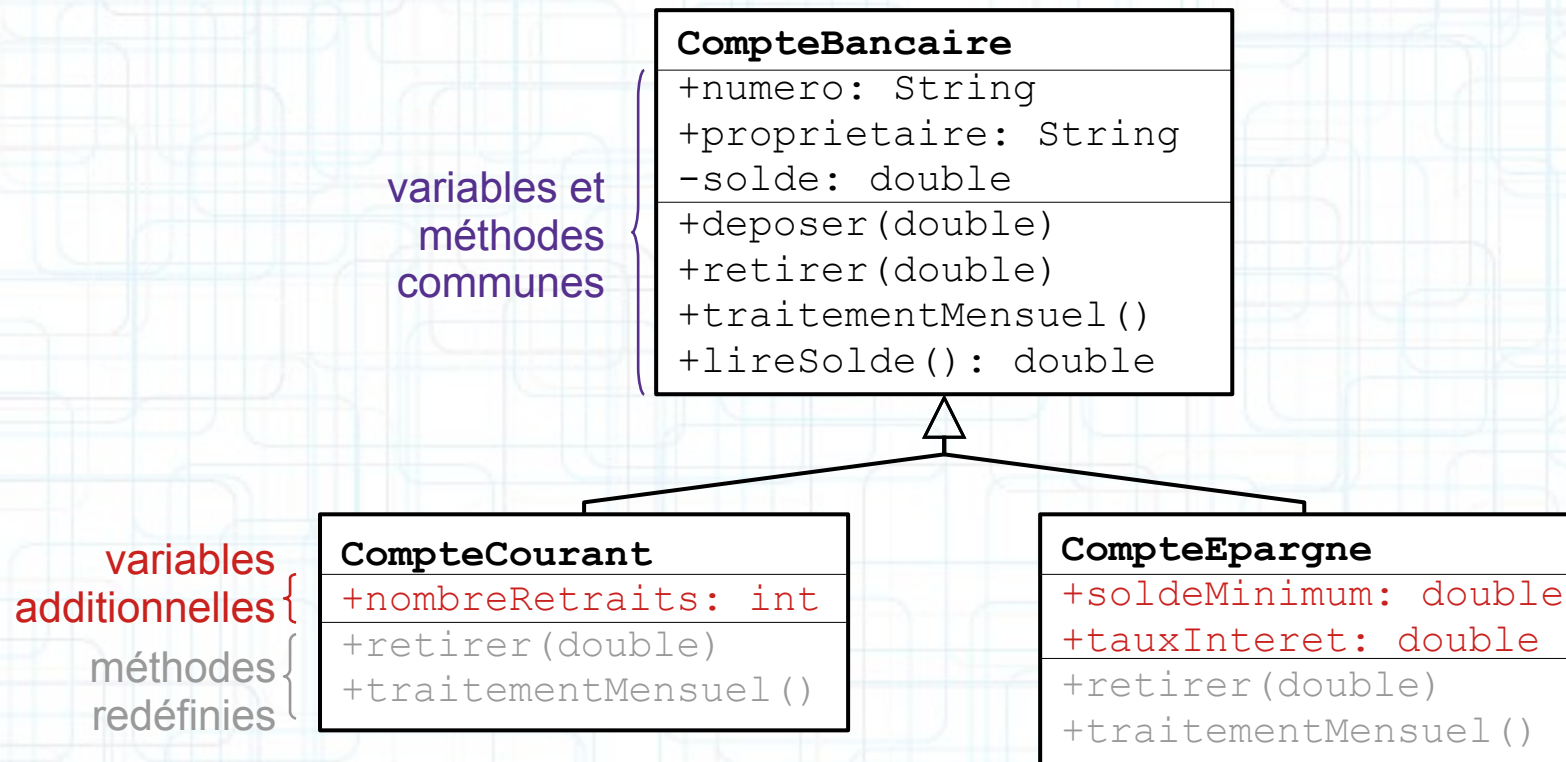
Late-binding → Enfant.setX(),
puis early-binding pour Enfant.x

Héritage

- **Exercice (suite)**
 - Modélisation de comptes bancaires
 - comptes épargnes : taux d'intérêt, appliqué mensuellement sur le solde minimum du mois
 - comptes courants : opérations de retrait payantes (N gratuites par mois)
 - structure commune : numéro de compte, propriétaire, solde
 - opérations communes : obtenir le solde, effectuer un dépôt, effectuer un retrait, appliquer un traitement mensuel (p.ex. calcul des intérêts)
 - Fournir une implémentation en Java tirant parti de l'héritage, de la redéfinition de méthode et du polymorphisme. Créer un tableau de comptes.

Héritage

- **Exercice (suite)**
 - Modélisation de comptes bancaires



Annexe

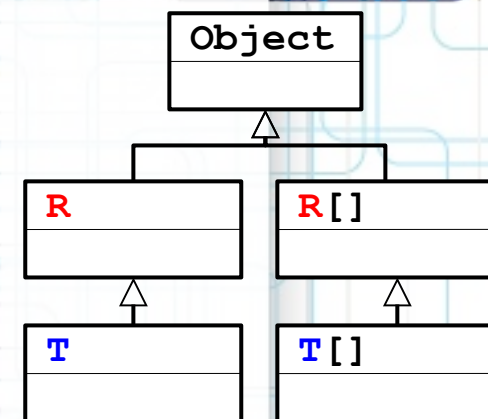
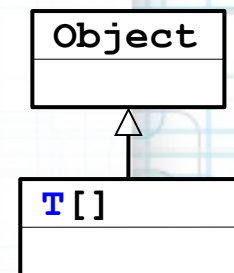
Héritage

- **Quid des tableaux ?**

- Les tableaux sont aussi des objets. Ils en héritent donc de la classe `Object` et son comportement. Quelque soit `T`, `T[]` $\triangleright$ `Object`. Ceci est aussi valable si `T` est primitif.

```
String[] tab = new String[3];  
System.out.println(tab.getClass().getName());  
[Ljava.lang.String;
```

- De plus, un héritage entre tableaux est possible. Ainsi si `T` et `R` sont des types objet tels que (`T` $\triangleright$ `R`), alors (`T[]` $\triangleright$ `R[]`). On dit que les tableaux d'objets sont **covariants**.



Héritage

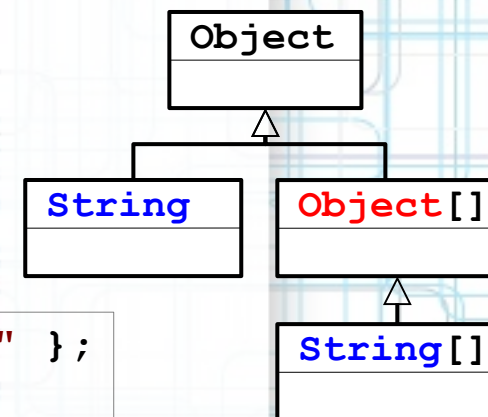
- **Quid des tableaux ?**

```
String[] auteurs = { "Orwell", "Bradbury", "Asimov" };
System.out.println(auteurs.toString());
```

```
Object obj = auteurs;
```

```
Object[] objects = auteurs;
System.out.println(objects.length);
System.out.println(objects);
```

[Ljava.lang.String;@74a14482



- **Danger des alias !** Le fait que (T[] ⊃ Object[]) pourrait « casser » la vérification statique de types de Java.

```
String[] auteurs = { "Orwell", "Bradbury", "Asimov" };
Object[] objects = auteurs;
objects[0] = new Carre(5, 7); /* accepté par le compilateur */
```

refusé par la JVM, ouf !

Exception in thread "main" java.lang.ArrayStoreException